NGINX

Stage 3

María Alonso León, Víctor Gil Bernal, Kimberly Casimiro Torres

Big Data
School of Computer Engineering (EII)
Las Palmas de Gran Canaria University, Spain

January 2025

1. Introduction

In modern distributed systems, load balancing has become a fundamental component to ensure the efficiency, availability, and scalability of applications handling a high volume of concurrent requests. As systems face increasing demands for processing and responsiveness, the need arises to implement mechanisms that distribute workloads evenly across various system components. This process, known as load balancing, is essential to avoid bottlenecks, prevent server overload, and ensure that end users experience smooth and fast interactions.

In this context, NGINX stands out as one of the most popular and effective tools for implementing load balancing in distributed systems. NGINX is a web server and reverse proxy capable of handling HTTP requests and functioning as a robust, high-performance load balancer. Its flexibility, scalability, and ease of configuration make it an ideal solution for systems requiring high availability and efficient response handling. NGINX not only distributes incoming client requests among available nodes but also effectively manages node failures by redirecting requests to operational instances, ensuring service continuity.

The use of NGINX in this project directly addresses the need to ensure that our system can efficiently handle multiple concurrent requests. In our design, the NGINX load balancer acts as the central point where all client requests converge. These requests are analyzed, and based on the established configuration, NGINX selects a system node to process them. Once the node receives the request, it performs the necessary operations and generates a response, which is then sent back to NGINX. Finally, NGINX transmits the response to the client, completing the request-response cycle transparently and efficiently.

The adopted architecture ensures that no single system node is overloaded, as NGINX evenly distributes requests among the available nodes. This approach not only optimizes system performance but also enhances its resilience. If a node fails or becomes unavailable, NGINX can dynamically redirect requests to other active nodes, ensuring that users do not experience service interruptions. This ability to handle failures and adapt dynamically to changes in node availability is crucial for building robust and reliable systems.

The choice of NGINX for this project is not only due to its technical capabilities but also its integration with modern technologies like Docker, which enables deploying the load balancer in a containerized environment. This facilitates its configuration, deployment, and scalability while ensuring that the system can run consistently across different environments. In this project, Docker provides an additional layer of flexibility, allowing new nodes to be added to the system without significant interruptions and enabling simple updates to NGINX's configuration to reflect these changes.

In summary, the load balancing implemented with NGINX not only optimizes the flow of requests within the system but also enables a scalable, resilient, and efficient architecture. This design ensures that the system is prepared to handle current traffic demands while adapting to potential future expansions. The implementation of NGINX as a load balancer in this project is, therefore, a strategic solution that combines technical efficiency with the ability to deliver a consistent and high-quality user experience.

2. Implementation of NGINX

The implementation of NGINX in this project has been a key process to ensure the efficient distribution of requests among the available nodes in the system. This implementation was carried out with a meticulous approach, leveraging NGINX's flexible configuration capabilities and its integration with modern technologies like Docker. The primary objective was to establish a robust load balancing system that not only distributed requests efficiently but also ensured high node availability, thereby optimizing the system's overall performance.

The first step in the implementation was configuring the NGINX file, known as nginx.conf, which defines how the load balancer should operate. In this file, a section called upstream was configured, listing the available backend nodes. Each node was identified by its IP address and port, representing the system instances responsible for processing requests. This section allows NGINX to know where to redirect the requests, laying the foundation for load balancing. It is worth noting that this list of nodes is easily extensible, meaning nodes can be added or removed as needed simply by updating the configuration file.

The server section of the nginx.conf file defines how NGINX should handle incoming requests. In this project, port 80 was configured to receive client requests. Within this section, the location block was configured to handle specific routes within the system, allowing requests to be redirected to specific

services or nodes based on the requested endpoints. This ensures a structured and precise management of requests. Additionally, a single additional header was included so that the nodes could correctly identify the original client and process the requests accurately.

To integrate NGINX into the project environment, Docker was used, a containerization tool that allows applications to run in isolated and consistent environments. A file named Dockerfile was created, defining how to build a custom Docker image for NGINX. This file includes instructions to copy the nginx.conf configuration file into the container, ensuring that NGINX uses the specific configuration for this project. Once the Dockerfile was configured, an automated script was used to build the Docker image and run the container within the network defined for the project. This approach, instead of manual commands, streamlines the deployment process and reduces potential errors.

The load balancing configured in NGINX distributes incoming requests among the available nodes. This means that when a client sends a request, it is initially received by NGINX, which then redirects it to one of the backend nodes defined in the configuration file. Once the node processes the request, it sends the response back to NGINX, which in turn relays it to the client. This flow ensures a smooth experience, as requests are processed efficiently, and response times remain low. Furthermore, NGINX monitors the status of the nodes and, if it detects that one is unavailable, automatically redirects requests to the remaining active nodes, ensuring continuous service availability.

In conclusion, the implementation of NGINX as a load balancer in this project has enabled the creation of a highly scalable and efficient system. From the detailed configuration of the nginx.conf file to its integration with Docker, each step was designed to optimize request handling and ensure efficient load distribution. This approach not only improves system performance but also provides the flexibility needed to adapt to future changes in the infrastructure. The combination of NGINX and Docker has proven to be a powerful solution to meet the demands of a modern distributed system, ensuring high-quality user experiences and the capability to handle growing workloads.

3. System Functionality

The implementation of NGINX in this project has been a key process to ensure the efficient distribution of requests among the available nodes in the system. This implementation was carried out with a meticulous approach, leveraging NGINX's flexible configuration capabilities and its integration with modern technologies like Docker. The primary objective was to establish a robust load balancing system that not only distributed requests efficiently but also ensured high node availability, thereby optimizing the system's overall performance.

The first step in the implementation was configuring the NGINX file, known as nginx.conf, which defines how the load balancer should operate. In this file, a section called upstream was configured, listing the available backend nodes. Each node was identified by its IP address and port, representing the system instances responsible for processing requests. This section allows NGINX to know where to redirect the requests, laying the foundation for load balancing. It is worth noting that this list of nodes is easily extensible, meaning nodes can be added or removed as needed simply by updating the configuration file.

The server section of the nginx.conf file defines how NGINX should handle incoming requests. In this project, port 80 was configured to receive client requests. Within this section, the location block was configured to handle specific routes within the system, allowing requests to be redirected to specific services or nodes based on the requested endpoints. This ensures structured and precise request management. Additionally, a single additional header was included so that the nodes could correctly identify the original client and process requests accurately.

To integrate NGINX into the project environment, Docker was used, a containerization tool that allows applications to run in isolated and consistent environments. A file named Dockerfile was created, defining how to build a custom Docker image for NGINX. This file includes instructions to copy the nginx.conf configuration file into the container, ensuring that NGINX uses the specific configuration for this project. Once the Dockerfile was configured, an automated script was used to build the Docker image and run the container within the network defined for the project. This approach, instead of manual commands, streamlines the deployment process and reduces potential errors.

The load balancing configured in NGINX distributes incoming requests among the available nodes. This means that when a client sends a request, it is initially received by NGINX, which then redirects it to one

of the backend nodes defined in the configuration file. Once the node processes the request, it sends the response back to NGINX, which in turn relays it to the client. This flow ensures a smooth experience, as requests are processed efficiently, and response times remain low. Furthermore, NGINX monitors the status of the nodes and, if it detects that one is unavailable, automatically redirects requests to the remaining active nodes, ensuring continuous service availability.

Figure 1 below presents a visual summary of the system's architecture. This diagram illustrates how NGINX acts as a load balancer, distributing requests among the backend nodes while managing communication with the storage components (datamart and datalake). This graphic helps to understand the overall implementation and its interaction with the different modules of the system.

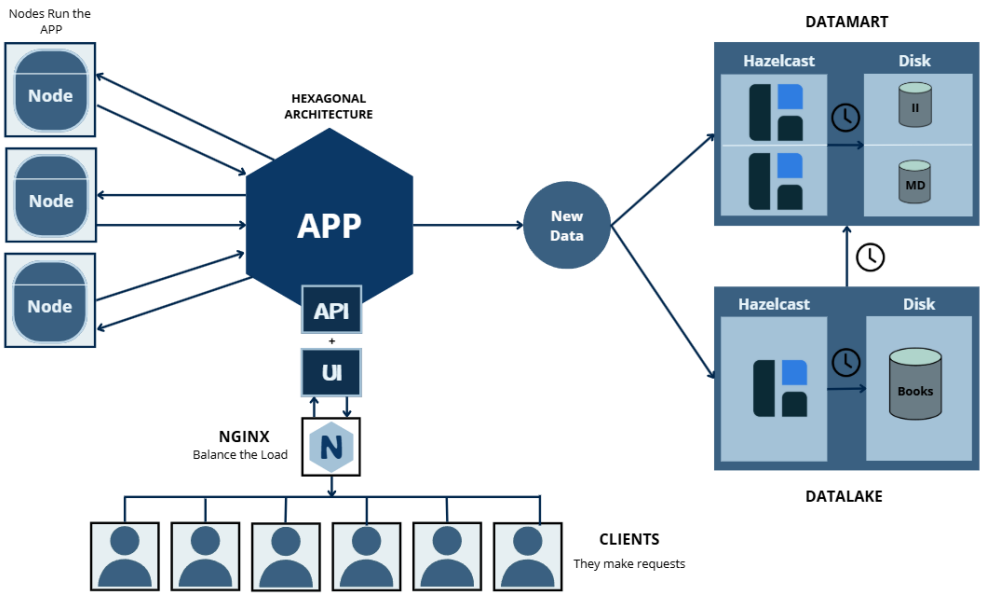

Figure 1: System Functionality Diagram with NGINX as Load Balancer

In summary, the implementation of NGINX as a load balancer in this project has enabled the creation of a highly scalable and efficient system. From the detailed configuration of the nginx.conf file to its integration with Docker, each step was designed to optimize request handling and ensure efficient load distribution. This approach not only improves system performance but also provides the flexibility needed to adapt to future changes in the infrastructure. The combination of NGINX and Docker has proven to be a powerful solution to meet the demands of a modern distributed system, ensuring high-quality user experiences and the capability to handle growing workloads.

4. Conclusion

The implementation of NGINX as a load balancer in this project has been fundamental to building a robust, scalable, and highly efficient system. This key component has proven capable of effectively managing the demands of a high-traffic environment by distributing requests efficiently among the backend nodes and ensuring that system resources are optimally utilized. NGINX's ability to centralize and manage interactions between clients and nodes not only simplifies the workflow but also adds an extra layer of security and abstraction that protects the integrity of the backend nodes.

The system's design, supported by NGINX, has enabled it to handle high volumes of traffic without compromising speed or reliability in responses. Requests are efficiently distributed among the nodes, preventing overload on a single point in the system. This ensures that each node contributes proportionally to processing requests, improving the overall efficiency of the system and providing consistent response times for users. Additionally, NGINX's capability to automatically redirect requests in case of node failures ensures the system remains operational even under contingency scenarios, reinforcing its high availability and reliability.

Another noteworthy aspect of this project is the system's readiness for future expansions. NGINX's

flexibility allows for the seamless integration of new backend nodes by updating its configuration without requiring significant changes to the overall architecture. This ensures that the system can easily adapt to changing traffic demands or incorporate advanced load-balancing configurations based on the project's needs. This adaptability makes the system not only effective in its current state but also well-prepared to tackle future challenges in an ever-evolving environment.

The integration of NGINX with Docker has been another significant advantage in this project, facilitating the creation of a reproducible and easily manageable deployment environment. This not only enables greater flexibility in development and testing but also ensures that the system can be deployed on diverse platforms with minimal additional configuration. Instead of relying on a set of manual commands, a script was implemented to automate the building of the Docker image and the execution of the container, streamlining the deployment process and reducing potential errors. This approach ensures that the system can scale horizontally quickly and efficiently, offering a modern solution aligned with current best practices.

In conclusion, the implementation of NGINX in this project represents a significant advancement in the creation of efficient and reliable distributed systems. Its ability to manage high volumes of traffic, distribute loads efficiently, and handle failures effectively makes it an indispensable tool for projects requiring high availability and performance. Additionally, its flexible and scalable design ensures that the system is prepared to handle future expansions and technological challenges, positioning it as a robust and sustainable solution in the field of distributed systems.